Where are we at with the Wasm?

About me



Mikkel Mørk Hegnhøj
I live in Boes (near Ry 30 minutes from Aarhus)

Product Management and Engineering at Microsoft, Fermyon, and Akamai.

Building developer focused tools, distributed systems, cloud services, and OSS projects.

K8s journey began around 1.17 (EOL 5 years ago)
Worked on WINDOWS CONTAINERS, Azure Kubernetes Services, and Virtual Kubelet
Spin Framework
Akamai Functions

How it started

Current status

What’s ahead for Wasm

How it started

The Pre-Wasm Era (Pre-2015)

- **2013**: asm.js (Mozilla) and PNaCl (Portable Native Client - Google) - Want to **bring native code to the browser**

Browser Specification & Launch (2015–2019)

- **2015**: WebAssembly (Wasm) is publicly announced as **a collaborative project** between Mozilla, Google, Microsoft, and Apple.
- **2017**: **Wasm MVP** (Minimum Viable Product) with native support is **shipped in all four major browsers**: Chrome, Firefox, Edge, and Safari.
- **2018**: The **Emscripten** toolchain reaches a mature state, allowing complex applications (like the Unity game engine) to compile to the web via Wasm. - https://silentspacemarine.com/
- **2019**: The W3C officially promotes WebAssembly to a Recommendation, making it t**he fourth official language** of the web alongside HTML, CSS, and JavaScript.

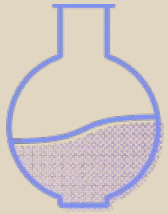
WebAssembly Design

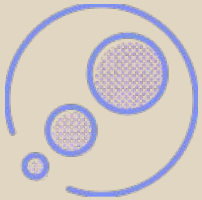
Portability

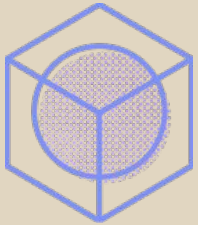
Small Package Sizes

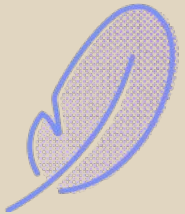
Sandboxed Execution

Fast Startup Times









WebAssembly (abbreviated Wasm[2]) is a *safe, portable, low-level code format* designed for efficient execution and compact representation. Its main goal is to enable high performance applications on the Web, but it does not make any Web-specific assumptions or provide Web-specific features, so it can be employed in other environments as well.

WASM outside the browser

Moving Outside the Browser (2019–2022)

- **2019**: The **Bytecode Alliance is formed** by Mozilla, Fastly, Intel, and Red Hat. **Wasmtime** released as a runtime for non-browser platforms.
- **2021**: **Cloudflare and Fastly** integrate Wasm into their Edge Computing platforms.
- **2021/22**: **wasmCloud** and **Spin Framework** introduced - developer tooling and hosting. Docker announces integrated support for WebAssembly.
- **2024**: **SpinKube released** - WebAssembly support in Kubernetes.
- **2025**: **Akamai** integrate Wasm into their Edge Computing platforms.

Enough about history - where are we today?

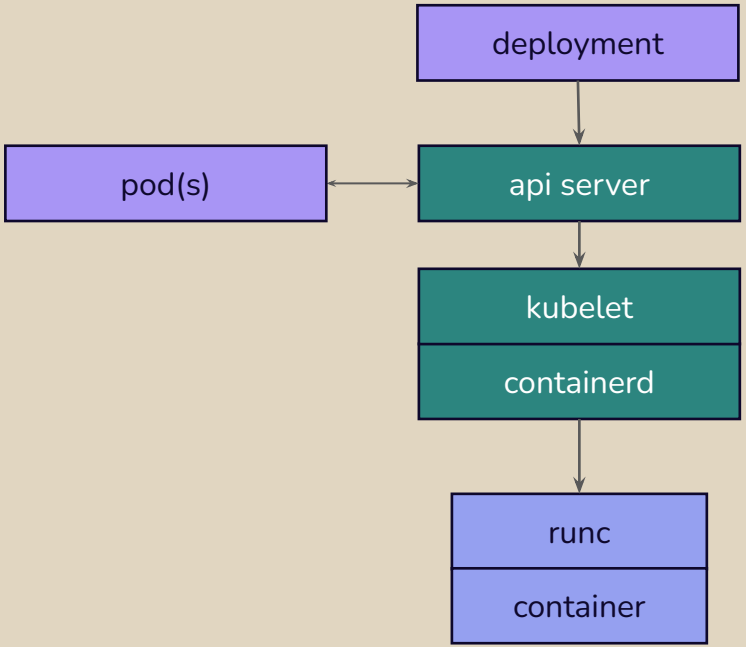
The Spin Framework

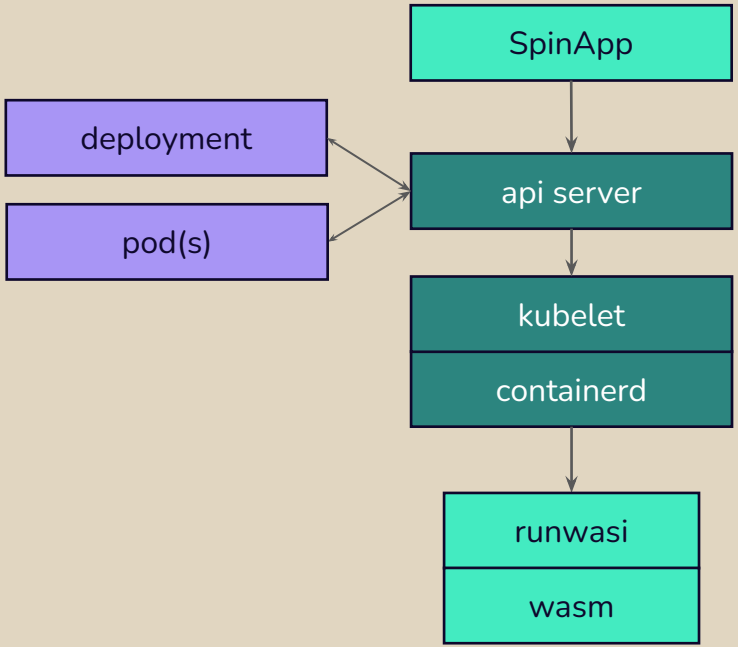
A framework for building and running event-driven microservice applications with WebAssembly (Wasm) components.

- Spin the Developer Tool
- The Spin runtime (CLI)
- SpinKube

https://spinframework.dev/

SpinKube


deployment
pod(s)
api server
kubelet
containerd
runc
container


SpinApp
deployment
pod(s)
api server
kubelet
containerd
runwasi
wasm

SpinKube - why isn’t everyone using this today?

Ecosystem maturity - not all apps can compile to Wasm

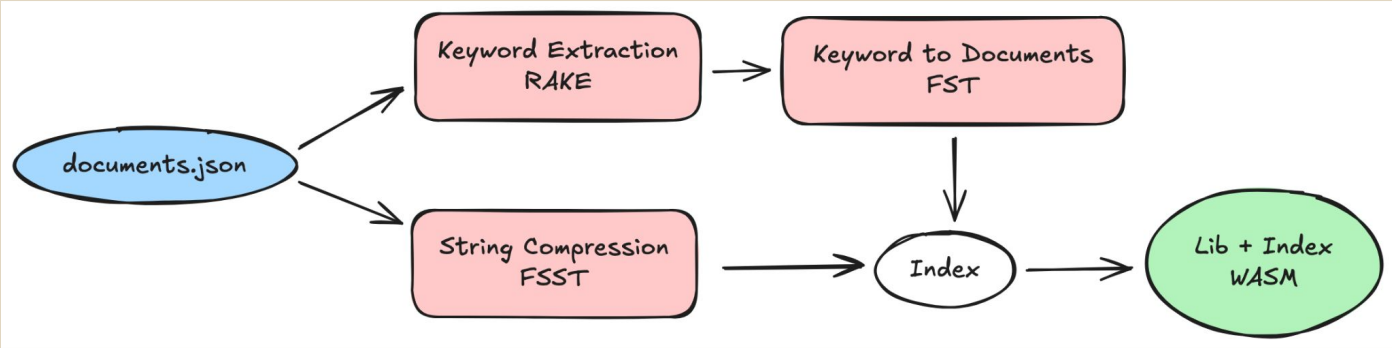
- Go support for WASI 0.2 and component model just landed
- Porting Node code to Wasm
- Refactor for serverless (handlers vs. web servers)

Utilization and scaling story - impacts the positive trade-off on utilization

- SpinApps to # of pods = 1:1 - You’ll run one instance of the Spin runtime for each SpinApp
- Scaling relies on controller feedback

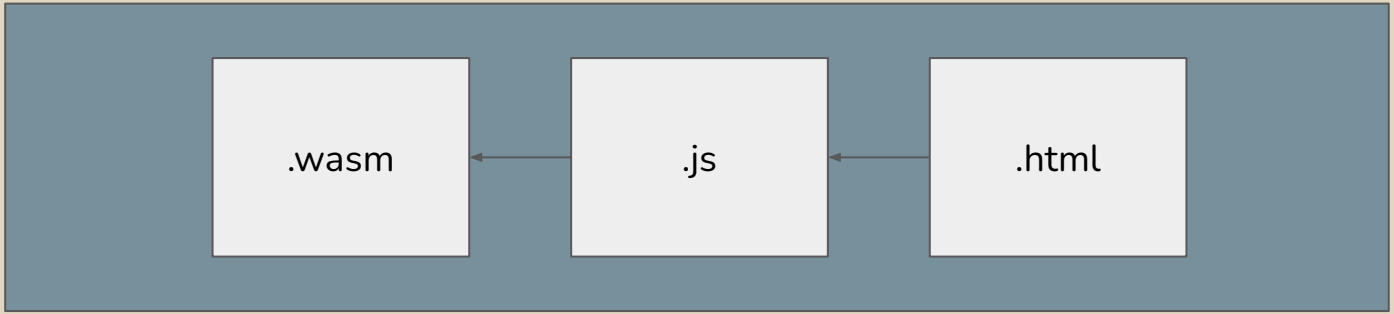
Docfind - Native code in the browser

Build

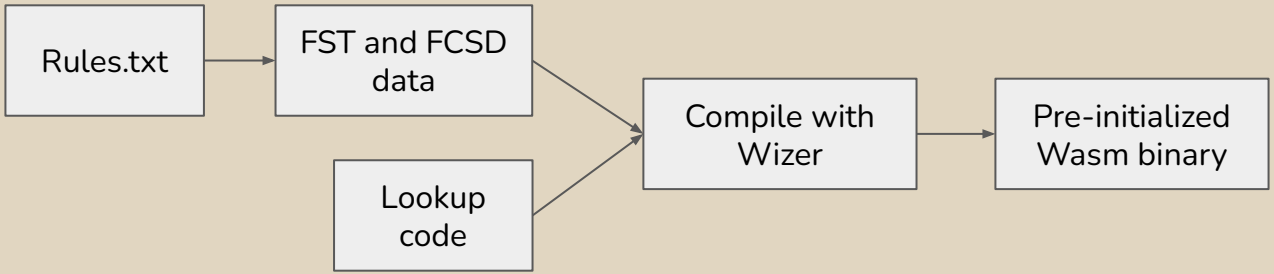


https://code.visualstudio.com/blogs/2026/01/15/docfind

Browser



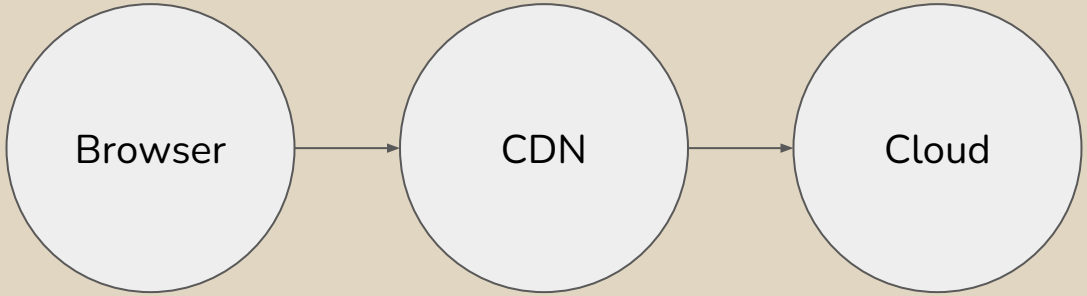
Large-scale-redirects



https://github.com/akamai-developers/akamai-functions-samples/tree/main/samples/large-scale-redirects

Edge (CDN)

- Deploy code to many locations close to users
  - Akamai Functions
  - Cloudflare Workers
  - Fastly Compute@Edge



Extensions

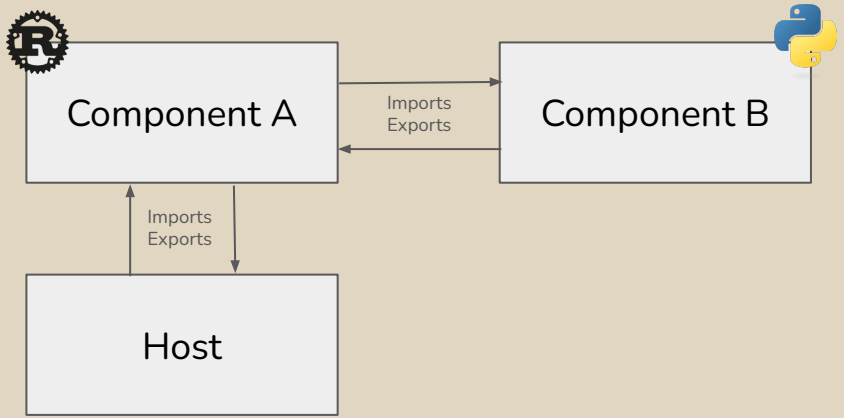
- Proxy-wasm - https://github.com/proxy-wasm
  - Proxy extension (filters) ABI specification
  - Envoy, Kong, NGINX, and more

- VS Code extensions - https://code.visualstudio.com/blogs/2024/05/08/wasm
  - Native performant extensions (LSPs)
  - Component Model

What’s ahead?

Component Model 1.0

Standardizing ABIs and well as guest-to-guest interfaces



https://component-model.bytecodealliance.org/

Things I’m looking forward to

1. Continued improvement in language support

2. Component Model 1.0 and WASI 1.0

3. AI Agentic Workloads

The Road to Component Model 1.0

Jun 8, 2026 Eric Gregory

WASI P3 is almost here, bringing native async support to the **WebAssembly System Interface (WASI)** and **Component Model**. In this post, we're looking to the *next* big milestone: a stable, formally specified Component Model 1.0.

https://bytecodealliance.org/articles/the-road-to-component-model-1-0

WASI 0.3 Launched

Jun 11, 2026 Bailey Hayes and Yosh Wuyts

WASI 0.3 is official, and async is now native to WebAssembly Components. The WASI Subgroup voted to ratify WASI 0.3.0, rebasing WASI onto the WebAssembly Component Model's async primitives. The 0.3.0 specification is now stable, and runtime and toolchain support is landing now.

https://bytecodealliance.org/articles/WASI-0.3

Thank you!